

Concept 9: Inheritance in Data Modeling (Héritage)

Why This Matters

When your boss said "use the reminder as a base model and specify through héritage (inheritance)," he's applying an object-oriented programming concept to data design. This pattern appears constantly in business applications where you have a general concept with specialized variants.

The Concept

Inheritance means defining a **base model** with shared properties, then creating **specialized models** that inherit everything from the base and add their own specific properties.

In Code (Object-Oriented)

python

```
# Base class – shared properties
class Reminder:
    id: int
    rfq_id: int
    trigger_date: datetime
    message: str
    status: str # pending, sent, cancelled
    created_by: int
    created_at: datetime

# Specialized class – inherits all of Reminder + adds its own
class InternalReminder(Reminder):
    assigned_to: int # which team member
    department: str # engineering, commercial, management
    priority: str # low, medium, high

# Another specialized class
class ExternalReminder(Reminder):
    client_contact_id: int # which client contact to notify
    communication_channel: str # email, phone, portal
    template_id: int # which notification template to use
```

`InternalReminder` and `ExternalReminder` both HAVE everything `Reminder` has (id, rfq_id, trigger_date, etc.) PLUS their own specific fields.

Mapping Inheritance to a Database

The challenge: databases don't have "inheritance" like code does. You need a strategy to represent this concept in tables. There are three common patterns:

Strategy 1: Single Table Inheritance (STI) ← Recommended for your case

One table holds ALL types, with a `type` discriminator column and nullable columns for type-specific fields.

reminder	
id (PK)	
type (discriminator: 'internal' 'external')	
rfq_id (FK)	
trigger_date	
message	
status	
created_by (FK)	
created_at	
— Internal-specific (NULL when type='external') —	
assigned_to (FK, nullable)	
department (nullable)	
priority (nullable)	
— External-specific (NULL when type='internal') —	
client_contact_id (FK, nullable)	
communication_channel (nullable)	
template_id (nullable)	

Pros:

- Simple — one table, no joins
- Easy to query all reminders regardless of type
- Easy to add new types (just add columns)
- Best performance for small number of type-specific fields

Cons:

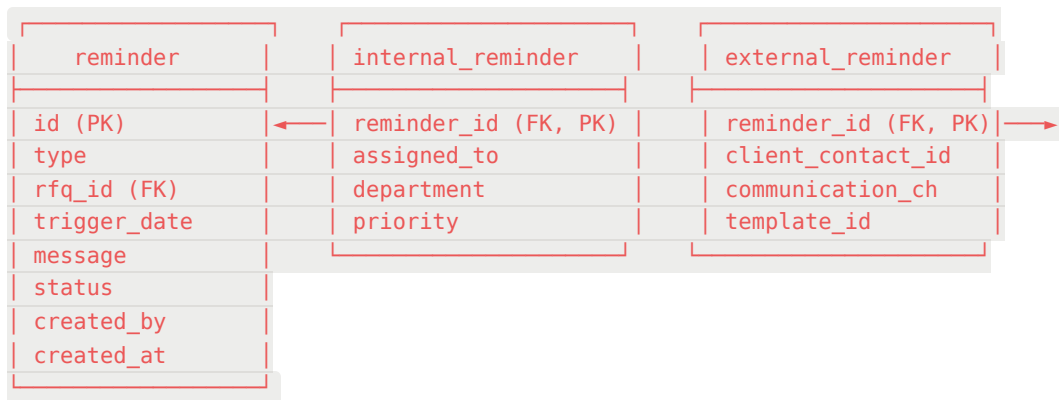
- Nullable columns (some fields are always NULL for certain types)

- Table can get wide if types diverge significantly
- No database-level enforcement that InternalReminder always has `assigned_to`

When to use: When the types share MOST of their fields and diverge only slightly. This is your case — Internal and External reminders are 80% the same.

Strategy 2: Table Per Type (Class Table Inheritance)

One shared table for base fields + separate tables for type-specific fields.



Pros:

- No nullable columns — clean schema
- Database-level NOT NULL constraints on type-specific fields
- Clear separation of concerns

Cons:

- Requires JOIN to get complete reminder data
- More complex queries
- More tables to manage

When to use: When types have many distinct fields and you want strict schema enforcement.

Strategy 3: Table Per Concrete Class

**Separate tables for each type, each containing ALL fields (shared + specific).
No base table.**

internal_reminder	external_reminder
id (PK)	id (PK)
rfq_id (FK)	rfq_id (FK)
trigger_date	trigger_date
message	message
status	status
created_by	created_by
created_at	created_at
assigned_to	client_contact_id
department	communication_channel
priority	template_id

Pros:

- No joins, no nulls
- Each table is self-contained

Cons:

- Duplicated columns across tables
- Can't easily query "all reminders" without UNION
- Schema changes to shared fields must be applied to ALL tables

When to use: When types are fundamentally different and rarely queried together.
NOT your case.

Recommendation for Your Reminder Entity

Go with Strategy 1 (Single Table Inheritance) because:

1. Internal and External reminders share ~80% of their fields
2. You'll often need to query "all reminders for this RFQ" regardless of type
3. The batch process needs to scan all reminders — one table is simpler
4. The number of type-specific fields is small (3-4 per type)
5. It's the simplest to implement and maintain

The `type` column becomes your discriminator, and your application code handles the logic of which fields apply to which type.

Beyond Reminders — Where Else This Pattern Appears

This inheritance pattern shows up everywhere in business applications:

- **User types:** `User` → `InternalUser` , `ExternalUser` (client portal)
- **Document types:** `Document` → `TechnicalSpec` , `CommercialOffer` , `PurchaseOrder`
- **Communication types:** `Communication` → `Email` , `PhoneCall` , `Meeting`
- **Notification types:** `Notification` → `SystemAlert` , `UserNotification` , `ReminderAlert`

Recognizing this pattern helps you design consistent models across your entire platform.

Key Vocabulary

Term	Definition
Base model	The parent class/table containing shared properties
Héritage (Inheritance)	The mechanism by which a specialized model receives all properties of a base model
Discriminator	A column (usually <code>type</code>) that indicates which specialized variant a record belongs to
STI (Single Table Inheritance)	One table for all types, with a discriminator column and nullable type-specific fields
Class Table Inheritance	One base table + separate tables for each type's specific fields, joined by FK
Polymorphism	The ability to treat different types uniformly through their shared base interface